VeriSight: Master Codebase & Architecture Journal

Comprehensive File-by-File Technical Guide & Implementation Reference for the Autonomous Silicon Verification Platform

Authors / System Architects: Patel Kittu & Vinod Nikhil Monterio • Mirafra Technologies • Release v2.4 Enterprise Edition

---

Executive Codebase Overview: This journal is a definitive technical companion to the VeriSight platform. It exhaustively documents the coding paths, architectural design patterns, algorithms, and data contracts implemented across the entire verisight/ package. VeriSight is architected around a **Tri-Engine design**: *Engine 1 (VeriSight Debug)* for autonomous root-cause analysis and zero-touch patching; *Engine 2 (VeriSight Plan)* for spec-driven feature decomposition, 50+ scaled test generation, and SVA suites; and *Engine 3 (VeriSight TestBench)* for spec-grounded UVM 1.2 synthesis enforcing the **Spec-as-Truth** principle. Every file below is documented with its exact repository path, key classes and functions, inter-agent dataflow, and practical logic explanation in simple English.

1. Platform Entry Points, Wizard Launcher & Orchestration

■ `File Path: verisight/wizard.py`

System Role: Interactive Terminal Guided Wizard & 2x2 Tri-Engine Launcher

Description & Purpose: The primary human-facing CLI launcher invoked by ./start. Displays modern 2x2 colorful interactive cards with silicon hero badges. It routes user choices cleanly into Engine 1 (VeriSight Debug), Engine 2 (VeriSight Plan), or Engine 3 (VeriSight TestBench). Crucially, in Engine 2, it implements pure spec-driven mode where RTL is completely optional—preventing default fallback hallucinations to ALU when only a specification PDF is provided.

Key Classes & Functions: VeriSightLauncher • display_banner() • display_menu() • handle_debug() • handle_plan() • handle_testbench() • run()

Interactions & Dataflow: Invokes verisight/orchestrator.py for Engine 1, verisight/plan/planner_orchestrator.py for Engine 2, and verisight/agents/spec_driven_tb_agent.py for Engine 3.

■ `File Path: verisight/orchestrator.py`

System Role: Multi-Agent Cognitive Pipeline Orchestrator (Engine 1)

Description & Purpose: The central control hub that sequences the 5 verification agents during an autonomous debugging session. It initializes data structures, passes strongly-typed JSON artifacts between agents, monitors step execution, and implements graceful offline fallback if an LLM API call fails. Ensures mathematical rigor from raw inputs to verified patches.

Key Classes & Functions: VeriSightOrchestrator • run_pipeline() • _run_agent1() • _run_agent2() • _run_agent3() • _run_agent4() • _run_agent5()

Interactions & Dataflow: Coordinates Agent 1 (Knowledge), Agent 2 (Classifier), Agent 3 (RTL Analyzer), Agent 4 (Reporter), and Agent 5 (Fix Generator).

■ File Path: verisight/config.py

System Role: Global Configuration, Model Providers & Environment Management

Description & Purpose: Manages global system configuration, simulator paths, API keys, and model provider routing. Allows seamless switching between Google Gemini 2.5 Flash, Anthropic Claude 3.5 Sonnet, OpenAI GPT-4o, and local air-gapped models via Ollama or vLLM (DeepSeek-Coder-V2, Llama-3-70B). Enforces strict zero-telemetry and security policies.

Key Classes & Functions: VeriSightConfig • ModelProvider • get_config() • validate_env() • resolve_model()

Interactions & Dataflow: Loaded by all agents and tools across the entire platform to establish runtime parameters.

■ File Path: verisight/__init__.py

System Role: Top-Level Package Initializer

Description & Purpose: Exposes package metadata, versioning (v2.4 Enterprise Edition), and core package-level imports for clean programmatic integration into third-party Python automation scripts and test harnesses.

Key Classes & Functions: __version__ • __author__ • __license__

Interactions & Dataflow: Package-level initialization file.

2. Specialized Multi-Agent Intelligence Fabric

■ File Path: verisight/agents/base_agent.py

System Role: Abstract Base Agent & LLM Gateway

Description & Purpose: The foundational abstract base class for all VeriSight agents. Encapsulates LLM invocation logic, exponential backoff retries, JSON schema enforcement via Pydantic, and automatic prompt token budgeting. It provides the essential fallback mechanism to deterministic offline rule engines when network or API issues occur.

Key Classes & Functions: BaseAgent • invoke_llm() • _call_gemini() • _call_claude() • _call_ollama() • _validate_schema() • retry_with_backoff()

Interactions & Dataflow: Inherited by Agent 1, Agent 2, Agent 3, Agent 4, Agent 5, and the Spec-Driven TB Agent.

■ File Path: verisight/agents/agent1_knowledge.py

System Role: Agent 1: Knowledge Engine & Multi-Modal Ingestion

Description & Purpose: The first stage in the triage pipeline. Ingests raw collateral: SystemVerilog RTL, UVM testbenches, simulation logs, and VCD waveforms. Uses pyslang to elaborate the hardware AST and pywellen to slice temporal waveform windows. Emits a clean, strongly-typed knowledge.json artifact.

Key Classes & Functions: KnowledgeAgent • extract_knowledge() • _parse_ast() • _tokenize_logs() • _slice_waveforms() • build_knowledge_graph()

Interactions & Dataflow: Receives raw user inputs; outputs knowledge.json consumed by Agent 2.

■ File Path: verisight/agents/agent2_classifier.py

System Role: Agent 2: Root Cause Classifier & 9-Rule Decision Engine

Description & Purpose: The diagnostic brain of VeriSight. Evaluates the 9 Formal Decision Rules against the extracted knowledge graph to mathematically classify whether a failure is an RTL Bug, Testbench Bug, Spec Mismatch, or Unknown. Calculates a Bayesian confidence score based on physical evidence (timestamps, signal values, AST lines). Emits summary.json.

Key Classes & Functions: ClassifierAgent • classify_failure() • _evaluate_rules() • _build_evidence_graph() • _calculate_confidence() • generate_summary()

Interactions & Dataflow: Consumes knowledge.json; outputs summary.json consumed by Agent 3, Agent 4, and Agent 5.

■ File Path: verisight/agents/agent3_rtl_analyzer.py

System Role: Agent 3: RTL Root Cause Analyzer & Logic Cone Traversal

Description & Purpose: Performs deep hardware inspection. Runs the static X-Tracer to catch uninitialized flip-flops, executes WaveTrace backwards cone traversal from failing output pins, inspects asynchronous clock-domain crossings (CDC) for missing 2-FF synchronizers, and detects FSM deadlocks. Emits rtl_analysis.json.

Key Classes & Functions: RTLAnalyzerAgent • analyze_rtl() • _run_xtracer() • _trace_cone() • _check_cdc() • _check_fsm_deadlocks()

Interactions & Dataflow: Consumes summary.json and knowledge.json; passes verified circuit evidence to Agent 4 and Agent 5.

■ File Path: verisight/agents/agent4_reporter.py

System Role: Agent 4: Interactive Reporter & Web Cockpit Compiler

Description & Purpose: Compiles all gathered diagnostic evidence into human-friendly and machine-readable deliverables. Generates the interactive HTML dashboard (report.html) featuring synchronized waveform inspection, executive summaries (summary.txt), technical Markdown reports (report.md), and CI/CD JSON artifacts (error.json).

Key Classes & Functions: ReporterAgent • generate_reports() • build_html_dashboard() • compile_markdown() • export_json_schemas()

Interactions & Dataflow: Consumes outputs from Agents 1, 2, 3, and 5 to create the final presentation deliverables.

■ File Path: verisight/agents/agent5_fix_generator.py

System Role: Agent 5: Code Fix Generator & Sandbox Verification

Description & Purpose: The automated repair agent. Operates under a strict 'minimal-code-region' policy to synthesize surgical git diffs targeting only the root-cause bug. It parses modified code through pyslang for syntax pre-checks, copies the design into an isolated /tmp sandbox, and re-executes simulation to guarantee regression sign-off.

Key Classes & Functions: FixGeneratorAgent • generate_fix() • _synthesize_patch() • _validate_in_sandbox() • _verify_regression()

Interactions & Dataflow: Consumes rtl_analysis.json; generates proposed.patch verified via verisight/tools/sim_runner.py.

■ File Path: verisight/agents/spec_driven_tb_agent.py

System Role: Engine 3: Spec-Driven UVM 1.2 Testbench Synthesis Agent

Description & Purpose: Implements the 10-Stage Reasoning Pipeline for complete UVM 1.2 testbench suite generation. Enforces the core 'Spec-as-Truth' principle: behavioral reference models and scoreboards are derived purely from architectural specifications, ensuring that RTL defects are independently detected rather than reproduced by the testbench. Generates 14 complete VIP components.

Key Classes & Functions: SpecDrivenTBAgent • run_spec_driven_pipeline() • _extract_spec_truth() • _generate_ref_model() • _synthesize_vip() • _validate_ieee1800()

Interactions & Dataflow: Invoked directly by verisight/wizard.py (Engine 3); interfaces with verisight/uvm/uvm_architect.py.

3. Engine 2 Deep Dive: Scaled Verification Planning & Coverage

■ File Path: verisight/plan/planner_orchestrator.py

System Role: Verification Plan Master Orchestrator

Description & Purpose: Orchestrates the entire verification planning lifecycle. Ingests design specifications (PDF, Markdown, text) and coordinates feature extraction, test planning, coverage architecture, and formal SVA generation. Handles optional RTL gracefully, allowing complete pre-RTL shift-left planning.

Key Classes & Functions: PlannerOrchestrator • generate_verification_plan() • _parse_spec() • _build_matrix() • _synthesize_assertions() • _export_artifacts()

Interactions & Dataflow: Coordinates feature_extractor.py, test_planner.py, coverage_architect.py, and assertion_architect.py.

■ File Path: verisight/plan/feature_extractor.py

System Role: Multi-Modal Specification Feature Decomposer

Description & Purpose: Parses architectural specifications (datasheets, PDFs, text) into atomic, numbered functional requirements (e.g. REQ_001, REQ_002). Extracts register maps, operational modes, timing parameters, and reset behaviors for downstream test matrix generation.

Key Classes & Functions: FeatureExtractor • extract_features() • _decompose_sections() • _extract_registers() • _extract_protocols()

Interactions & Dataflow: Feeds extracted atomic feature schemas to test_planner.py and coverage_architect.py.

■ File Path: verisight/plan/test_planner.py

System Role: 50+ Scaled Test Matrix Synthesizer

Description & Purpose: Generates over 50 unique, highly targeted verification testcases across 8 distinct categories: Directed, Constrained-Random, Reset/Initialization, Boundary/Corner, Error/Negative, Performance/Backpressure, Concurrency, and Protocol-Specific. Guarantees comprehensive functional coverage scaling.

Key Classes & Functions: TestPlanner • generate_tests() • _generate_category_tests() • _enforce_diversity() • _synthesize_sequences()

Interactions & Dataflow: Consumes atomic requirements from feature_extractor.py; outputs test_plan.json and test_plan.md.

■ File Path: verisight/plan/coverage_architect.py

System Role: Multi-Covergroup & Functional Coverage Model Synthesizer

Description & Purpose: Synthesizes 90+ coverage points and 150+ bins. Automatically generates SystemVerilog covergroups for state machines, illegal state avoidance, register bitfields, and cross-coverage between control and data signals, formatted ready for integration into UVM subscriber components.

Key Classes & Functions: CoverageArchitect • build_coverage_model() • _create_covergroups() • _define_bins() • _generate_cross_coverage()

Interactions & Dataflow: Outputs coverage_model.sv and coverage_spec.json for testbench integration.

■ File Path: verisight/plan/assertion_architect.py

System Role: Formal SystemVerilog Assertion (SVA) Synthesizer

Description & Purpose: Translates natural language protocol rules and timing constraints into formal concurrent SVA properties. Employs correct implication operators (|->, |=>), clocking blocks, and reset disable guards. Generates 30+ production assertions in a standalone bindable module.

Key Classes & Functions: AssertionArchitect • generate_assertions() • _create_sva_property() • _bind_clock_reset() • _generate_bind_file()

Interactions & Dataflow: Outputs formal SVA assertions file (assertions.sv) bindable to DUT instances.

■ File Path: verisight/plan/assertion_generator.py

System Role: Immediate & Dynamic Assertion Helper

Description & Purpose: Generates immediate assertions and procedural runtime checkers for internal bus interfaces, FIFO pointers, and arithmetic boundary conditions, complementing concurrent SVA properties.

Key Classes & Functions: AssertionGenerator • generate_immediate_checks() • create_protocol_checkers()

Interactions & Dataflow: Helper module utilized by assertion_architect.py.

■ File Path: verisight/plan/coverage_ingest.py

System Role: Simulator UCDB & vDB Coverage Database Parser

Description & Purpose: Parses Questa UCDB XML/text exports and Synopsys VCS vDB reports. Extracts actual simulation hit counts and coverage percentages for every covergroup and coverpoint.

Key Classes & Functions: CoverageIngest • ingest_ucdb() • ingest_vdb() • _parse_xml_report() • _extract_hit_counts()

Interactions & Dataflow: Feeds actual simulator coverage data to coverage_correlator.py.

■ File Path: verisight/plan/coverage_correlator.py

System Role: Dynamic Coverage Hole Analyzer & Directed Stimulus Generator

Description & Purpose: Compares planned architectural covergroups against actual simulator hit counts. Highlights zero-hit bins in red and automatically synthesizes directed UVM sequence constraints specifically targeting rare corner cases, turning 6-8 weeks of random iterations into days.

Key Classes & Functions: CoverageCorrelator • correlate() • _find_unhit_bins() • generate_directed_stimulus()

Interactions & Dataflow: Consumes planned models and live UCDB data; outputs directed UVM test sequences.

■ File Path: verisight/plan/traceability.py

System Role: ISO 26262 ASIL-D & DO-254 Traceability Matrix Exporter

Description & Purpose: Builds a bidirectional requirement-to-test traceability matrix linking every specification clause directly to its testcase, coverpoint, and SVA assertion. Exports audit-ready CSV (traceability_matrix.csv) and HTML reports for automotive safety sign-off.

Key Classes & Functions: TraceabilityMatrix • build_matrix() • export_csv() • export_html() • verify_completeness()

Interactions & Dataflow: Outputs compliance artifacts required for functional safety certification.

■ File Path: verisight/plan/protocol_architect.py

System Role: Standard Protocol FSM & Handshake Decomposer

Description & Purpose: Models standard bus protocol rules (AMBA AXI, AHB, APB, I2C, SPI). Defines valid handshake latencies, backpressure boundaries, and illegal state transitions used by the test planner and SVA architect.

Key Classes & Functions: ProtocolArchitect • extract_protocol_states() • model_handshake() • get_timing_windows()

Interactions & Dataflow: Provides protocol domain rules to test_planner.py and assertion_architect.py.

■ File Path: verisight/plan/schemas.py

System Role: Pydantic Data Contracts for Verification Planning

Description & Purpose: Defines strict Pydantic schemas validating all verification planning artifacts. Ensures every requirement, testcase, and assertion maintains consistent naming, category tags, and bidirectional linkages.

Key Classes & Functions: PlanRequest • RequirementItem • TestCaseItem • CoverpointItem • AssertionItem • VerificationPlanOutput

Interactions & Dataflow: Enforces data structure contracts across all verisight/plan/ modules.

4. Engine 3 Deep Dive: Spec-Driven UVM 1.2 Synthesis

■ File Path: verisight/uvm/uvm_architect.py

System Role: Complete 14-Component UVM 1.2 Suite Synthesizer

Description & Purpose: The core code generator for Engine 3. Synthesizes all 14 files comprising a production-grade UVM 1.2 environment: Interface, Sequence Item, Sequencer, Driver, Monitor, Agent, Golden Reference Model, Scoreboard, Coverage Subscriber, Sequences, Environment, Base Test, Package, and Top-level testbench module.

Key Classes & Functions: UVMArchitect • synthesize_suite() • _generate_interface() • _generate_driver() • _generate_monitor() • _generate_reference_model() • _generate_scoreboard() • _generate_tb_top()

Interactions & Dataflow: Invoked by spec_driven_tb_agent.py; uses matrix_synthesizer.py for transaction definitions.

■ File Path: verisight/uvm/matrix_synthesizer.py

System Role: Protocol Matrix & Transaction Decomposer

Description & Purpose: Decomposes bus transactions into SystemVerilog sequence item fields with protocol constraints (e.g. address ranges, burst lengths, payload arrays). Generates rand fields, pre/post_randomize hooks, and print/compare methods.

Key Classes & Functions: MatrixSynthesizer • build_transaction_matrix() • derive_fields() • synthesize_constraints()

Interactions & Dataflow: Feeds transaction field models to uvm_architect.py for _seq_item.sv and _driver.sv synthesis.

■ File Path: verisight/uvm/__init__.py

System Role: UVM Synthesis Subsystem Initializer

Description & Purpose: Exposes the core UVM architecture and synthesis classes for package-level access.

Key Classes & Functions: UVMArchitect • MatrixSynthesizer

Interactions & Dataflow: UVM package initialization file.

5. Parsers, AST Compilers & Waveform Slicing

■ File Path: verisight/parsers/pyslang_rtl.py

System Role: IEEE-1800 SystemVerilog RTL Compiler & AST Elaborator

Description & Purpose: The compiler core of VeriSight. Uses pyslang (Slang C++ bindings) to elaborate genuine SystemVerilog ASTs. Extracts module port hierarchies, clock/reset sensitivity lists, and static reset assignments. Enables zero-simulation uninitialized register detection and AST logic cone pruning.

Key Classes & Functions: PySlangRTLParser • parse_rtl() • get_ast() • extract_ports() • find_uninitialized_regs() • trace_fanin()

Interactions & Dataflow: Provides elaborated AST representations to Agent 1, Agent 3, and the X-Tracer engine.

■ File Path: verisight/parsers/pyslang_uvm.py

System Role: SystemVerilog UVM Class Hierarchy & TLM Parser

Description & Purpose: Parses complex object-oriented UVM testbenches using pyslang. Resolves class inheritance (e.g. uvm_driver extends uvm_component), phase execution methods (build_phase, run_phase), virtual interfaces, and TLM 2.0 analysis ports.

Key Classes & Functions: PySlangUVMParser • parse_uvm() • extract_classes() • resolve_phases() • map_tlm_ports()

Interactions & Dataflow: Enables Agent 1 and Agent 2 to inspect and classify testbench-side code structures.

■ File Path: verisight/parsers/vcd_parser.py

System Role: High-Speed Rust-Backed Temporal Waveform Slicer

Description & Purpose: High-performance waveform parser powered by pywellen and vcdvcd. Solves the 50GB waveform memory problem by extracting only a 20-clock-cycle window surrounding the failure timestamp for active logic cone signals. Compresses gigabytes of trace data into a lightweight JSON slice.

Key Classes & Functions: VCDParser • parse_vcd() • slice_window() • get_signal_transitions() • find_x_timestamp()

Interactions & Dataflow: Provides temporal transition data to Agent 1, Agent 3, and the Web Waveform Studio.

■ File Path: verisight/parsers/log_parser.py

System Role: Simulation Log Tokenizer & Error Pattern Classifier

Description & Purpose: Tokenizes raw simulation logs from Synopsys VCS, Siemens Questa, Cadence Xcelium, Verilator, and Icarus Verilog. Automatically extracts UVM_ERROR strings, assertion failure timestamps, scoreboard mismatch payloads, and fatal tool crashes.

Key Classes & Functions: LogParser • parse_log() • extract_errors() • tokenize_fatal() • identify_simulator()

Interactions & Dataflow: Feeds tokenized error signatures to Agent 1 and Agent 2 for failure triage.

■ `File Path: verisight/parsers/spec_parser.py`

System Role: Natural Language Specification Document Ingester

Description & Purpose: Ingests architectural specifications from PDF, Markdown, and text files. Cleans up formatting, parses register tables, extracts timing diagrams, and outputs normalized text blocks for feature extraction.

Key Classes & Functions: SpecParser • parse_spec() • extract_markdown() • extract_pdf_text() • clean_tables()

Interactions & Dataflow: Used by Agent 1 and verisight/plan/feature_extractor.py to read design specs.

■ `File Path: verisight/parsers/coverage_parser.py`

System Role: General Coverage Report Tokenizer

Description & Purpose: Parses text-based coverage summary reports from various simulators, providing quick baseline statistics on line, branch, toggle, and functional coverage.

Key Classes & Functions: CoverageParser • parse_report() • extract_summary()

Interactions & Dataflow: Helper parser for quick coverage metrics extraction.

■ `File Path: verisight/parsers/rtl_parser.py`

System Role: Lexical SystemVerilog Fallback Tokenizer

Description & Purpose: Lightweight regex-based fallback tokenizer for quick scans and environments where Slang shared libraries cannot be loaded.

Key Classes & Functions: RTLParser • extract_modules() • extract_signals_regex()

Interactions & Dataflow: Provides secondary fallback for RTL parsing.

■ `File Path: verisight/parsers/uvm_parser.py`

System Role: Lexical UVM Fallback Tokenizer

Description & Purpose: Lightweight regex-based fallback tokenizer for rapid identification of UVM macros and class definitions.

Key Classes & Functions: UVMParser • extract_uvm_components()

Interactions & Dataflow: Provides secondary fallback for UVM parsing.

■ `File Path: verisight/parsers/sv_frontend.py`

System Role: Unified SystemVerilog Frontend Selector

Description & Purpose: Dynamically selects the best available parser frontend (preferring pyslang, falling back gracefully to lexical tokenizers if needed).

Key Classes & Functions: SVFrontend • parse() • get_best_parser()

Interactions & Dataflow: Unified interface for all SystemVerilog parsing requests.

6. Hardware Execution, Simulation Sandbox & Tools

■ File Path: verisight/tools/sim_runner.py

System Role: Closed-Loop Multi-Simulator Runner & Isolated Sandbox

Description & Purpose: Manages simulation execution inside an isolated /tmp sandbox workspace. Compiles design files, monitors timeout limits, captures stdout/stderr, and prevents broken or hallucinated AI patches from touching the user's primary git repository.

Key Classes & Functions: SimRunner • run_simulation() • create_sandbox() • execute_iverilog() • execute_verilator() • cleanup_sandbox()

Interactions & Dataflow: Used by Agent 5 for fix verification and by the CLI launcher for regression testing.

■ File Path: verisight/tools/eda_runner.py

System Role: Commercial EDA Simulator Adapter (VCS, Questa, Xcelium)

Description & Purpose: Enterprise wrapper for commercial EDA simulation engines. Executes Synopsys VCS, Siemens Questa, and Cadence Xcelium runs, extracting simulation artifacts, coverage databases, and FSDB/VCD waveforms without altering user Makefiles.

Key Classes & Functions: EDARunner • run_vcs() • run_questa() • run_xcelium() • wrap_makefile()

Interactions & Dataflow: Enables seamless Day-1 commercial EDA pipeline integration.

■ File Path: verisight/tools/sv_validate.py

System Role: Static IEEE-1800 Compiler Syntax Validator

Description & Purpose: Validates generated SystemVerilog code against IEEE-1800 standards using pyslang diagnostics. Serves as a strict compiler gateway: any syntax error in generated patches or UVM suites triggers a self-correction loop before code is presented to the user.

Key Classes & Functions: SVValidator • validate_syntax() • check_compilation() • get_slang_diagnostics()

Interactions & Dataflow: Gatekeeper invoked by Agent 5 and Engine 3 before saving code.

■ File Path: verisight/tools/xtracer_runner.py

System Role: X-Tracer Engine & Netlist Back-Tracing Driver

Description & Purpose: Coordinates static and dynamic X-propagation analysis. Invokes pyslang for reset scans and drives netlist back-tracing from failing signal cycles to isolate originating uninitialized flip-flops.

Key Classes & Functions: XTracerRunner • run_xtracer() • trace_origin_flop() • parse_xtracer_output()

Interactions & Dataflow: Invoked by Agent 3 during RTL failure analysis.

■ File Path: verisight/tools/traceweave.py

System Role: WaveTrace Backward Cone Traversal Engine

Description & Purpose: Implements DAG fan-in cone traversal. Given an output pin and cycle T, it traces through combinatorial gates, multiplexer select lines, and sequential stages to construct the exact circuit cone responsible for the signal's value.

Key Classes & Functions: TraceWeave • backward_cone_traversal() • get_driving_signals() • build_cone_graph()

Interactions & Dataflow: Used by Agent 3 to prune irrelevant signals and isolate bug origins.

■ File Path: verisight/tools/fault_localization.py

System Role: Statistical & Spectrum-Based Fault Localizer

Description & Purpose: Applies spectrum-based fault localization algorithms (like Tarantula and Ochiai) across simulation logs and coverage data to rank source code lines by their statistical probability of containing the bug.

Key Classes & Functions: FaultLocalizer • localize() • calculate_tarantula_score() • rank_suspicious_lines()

Interactions & Dataflow: Assists Agent 2 and Agent 3 in prioritizing candidate bug regions.

■ File Path: verisight/tools/reverse_debugger.py

System Role: Temporal Reverse Simulation Stepper

Description & Purpose: Allows virtual reverse-debugging through parsed VCD waveform traces, stepping backward cycle-by-cycle from failure points to examine register states prior to assertion violations.

Key Classes & Functions: ReverseDebugger • step_backward() • inspect_state_at_cycle()

Interactions & Dataflow: Utilized by Agent 3 and the interactive Web Waveform Studio.

■ File Path: verisight/tools/formal_verifier.py

System Role: Formal Property & Assertion Checker

Description & Purpose: Hooks into open-source formal tools (SymbiYosys / Yosys) to perform formal property verification on synthesized SVA assertions, proving properties mathematically over bounded cycles.

Key Classes & Functions: FormalVerifier • verify_property() • run_sby()

Interactions & Dataflow: Optional formal verification engine for SVA properties.

■ File Path: verisight/tools/yosys_runner.py

System Role: Yosys Open-Source Synthesis Driver

Description & Purpose: Executes Yosys synthesis scripts to generate gate-level netlists from RTL, supporting gate-level X-tracing and formal verification passes.

Key Classes & Functions: YosysRunner • synthesize_netlist() • elaborate_design()

Interactions & Dataflow: Netlist generation utility.

■ File Path: verisight/tools/eda_playground_exporter.py

System Role: Cloud EDA Playground 1-Click Exporter

Description & Purpose: Invoked by `./start edaplayground`. Packages synthesized UVM testbenches, RTL files, and run scripts into a single clean bundle and generates a shareable web link to edaplayground.com for instant remote sharing.

Key Classes & Functions: EDAPlaygroundExporter • package_project() • generate_share_link() • export_bundle()

Interactions & Dataflow: Platform utility for 1-click cloud sharing.

7. Type-Safe Data Contracts & Pydantic Schemas

■ File Path: verisight/schemas/knowledge_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Agent 1 output schema: contains parsed AST hierarchies, tokenized log errors, and temporal waveform slices.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/classification.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Agent 2 output schema: formal rule matches, RTL vs TB vs Spec classification, and Bayesian confidence scores.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/rtl_analysis.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Agent 3 output schema: static X-origins, WaveTrace fan-in cones, CDC hazards, and FSM deadlock states.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/fix_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Agent 5 output schema: surgical unified git patch, target file locations, and sandbox validation outcomes.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/report_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Agent 4 output schema: unified diagnostic report model powering report.html, report.md, and error.json.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/spec_driven_tb_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Engine 3 schema: 10-stage spec-driven UVM synthesis models, 14 VIP component manifests, and scoreboards.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/log_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Normalized data contract for simulation error tokens, timestamps, and simulator metadata.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/vcd_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Data contract for temporal waveform slices, signal transition tables, and 4-state logic dumps.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/rtl_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Data contract representing SystemVerilog module ports, parameters, interfaces, and clock domains.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/tb_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Data contract representing UVM testbench component topologies and TLM port connections.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/spec_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Data contract representing extracted architectural specification requirements.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

■ File Path: verisight/schemas/parser_schema.py

System Role: Pydantic Strongly-Typed Schema Contract

Description & Purpose: Common intermediate schemas for parser frontends and compiler elaborators.

Key Classes & Functions: BaseModel • Field • validate()

Interactions & Dataflow: Enforces strict data validation between agents, preventing hallucinated or malformed data propagation.

8. Live Web Studio, API Server & Waveform Telemetry

■ File Path: verisight/server/app.py

System Role: FastAPI Asynchronous Backend Server & WebSocket Bridge

Description & Purpose: High-performance FastAPI web backend launched by `./start serve`. Provides REST APIs for triggering triage sessions and bi-directional WebSockets for real-time simulation log streaming and dynamic progress reporting.

Key Classes & Functions: app • websocket_endpoint() • get_status() • stream_simulation_logs() • serve_static()

Interactions & Dataflow: Serves the frontend static assets and connects to verisight/orchestrator.py.

■ File Path: verisight/server/static/app.js

System Role: Interactive Browser Cockpit Frontend Logic

Description & Purpose: Vanilla JavaScript client application managing real-time WebSocket events, live coverage bar updates, interactive patch diff viewing, and waveform cursor synchronization.

Key Classes & Functions: initWebSocket() • renderDashboard() • updateCoverageBars() • syncWaveformCursor()

Interactions & Dataflow: Runs in the user's browser; interacts with FastAPI backend.

■ File Path: verisight/server/static/index.html

System Role: Live Web Studio Cockpit Single-Page UI

Description & Purpose: Modern single-page HTML interface structured into Live Coverage Cockpit, RCA Dashboard, Sub-Cycle Waveform Studio, and Surgical Patch Review.

Key Classes & Functions: #triage-view • #coverage-cockpit • #waveform-studio • #patch-viewer

Interactions & Dataflow: Loaded by browser clients accessing http://localhost:8000.

■ `File Path: verisight/server/static/style.css`

System Role: Sage / Cream / Burgundy Enterprise Theme Stylesheet

Description & Purpose: Custom stylesheet defining the elegant, professional Sage/Cream/Burgundy palette, dark-mode styling, responsive card grids, and high-contrast typography.

Key Classes & Functions: :root • .badge • .card-grid • .coverage-bar • .waveform-canvas

Interactions & Dataflow: Styles the Live Web Studio UI.

■ `File Path: verisight/reporter/waveform_studio.py`

System Role: Sub-Cycle Waveform Canvas & Cursor Synchronizer

Description & Purpose: Generates interactive browser waveform views. Synchronizes log error clicks with the exact physical assertion failure clock cycle (e.g. 450ns), decoding binary, hex, and one-hot FSM states dynamically.

Key Classes & Functions: WaveformStudio • render_canvas() • correlate_cursor() • format_radix()

Interactions & Dataflow: Embedded into report.html and the Live Web Studio.

9. Enterprise Security, Sovereign Air-Gap & Memory

■ `File Path: verisight/utils/airgap.py`

System Role: Sovereign Air-Gapped Verification & Local Endpoints

Description & Purpose: Enforces 100% offline air-gapped operation. Disables outbound HTTP requests, configures local Ollama or vLLM endpoints (DeepSeek-Coder, Llama-3), and ensures proprietary RTL never leaves the company firewall.

Key Classes & Functions: AirGapManager • verify_offline() • check_local_ollama() • block_external_network()

Interactions & Dataflow: Guarantees security for defense, aerospace, and high-security fabless semiconductor teams.

■ `File Path: verisight/utils/anonymizer.py`

System Role: Semiconductor RTL & Spec IP Sanitizer

Description & Purpose: Sanitizes proprietary chip IP when using commercial cloud APIs. Scrambles module names, signal identifiers, and comments before dispatch, and maps them back locally upon receiving the response.

Key Classes & Functions: Anonymizer • anonymize_code() • deanonymize_response() • scramble_identifiers()

Interactions & Dataflow: Protects customer IP during cloud LLM operations.

■ `File Path: verisight/utils/evidence_verifier.py`

System Role: Mathematical Evidence Proof & Verification Gatekeeper

Description & Purpose: Enforces that every diagnosis cites physical evidence: matching simulation timestamps, physical waveform transitions, and elaborated AST line numbers before classification is approved.

Key Classes & Functions: EvidenceVerifier • verify_evidence_chain() • validate_timestamps()

Interactions & Dataflow: Guardrail against AI hallucination.

■ File Path: verisight/utils/key_validator.py

System Role: API Key & Simulator License Pre-Flight Validator

Description & Purpose: Performs pre-flight checks during `./start` launcher initialization, verifying active API keys and local EDA simulator license availability.

Key Classes & Functions: KeyValidator • validate_gemini_key() • validate_claude_key() • check_simulator_licenses()

Interactions & Dataflow: Invoked during wizard startup.

■ File Path: verisight/utils/file_utils.py

System Role: Robust File I/O & Temporary Sandbox Manager

Description & Purpose: Provides atomic file operations, safe directory creation, and clean sandbox teardown utilities.

Key Classes & Functions: safe_read() • safe_write() • create_temp_dir() • atomic_write()

Interactions & Dataflow: General utility used throughout the platform.

■ File Path: verisight/utils/logger.py

System Role: Structured ANSI Color & Rotating File Logger

Description & Purpose: Configures structured logging with terminal color formatting and rotating file handlers for audit tracking.

Key Classes & Functions: VeriSightLogger • get_logger() • setup_logging()

Interactions & Dataflow: System-wide logging infrastructure.

■ File Path: verisight/utils/sv_lexer.py

System Role: SystemVerilog Tokenizer & Reserved Keyword Lexer

Description & Purpose: Fast lexical scanner identifying SystemVerilog keywords, operators, and literals.

Key Classes & Functions: SVLexer • tokenize() • is_keyword()

Interactions & Dataflow: Tokenization helper.

■ File Path: verisight/rag/knowledge_base.py

System Role: ChromaDB 11-Collection Vector Memory Store

Description & Purpose: Manages 11 persistent ChromaDB collections storing specification paragraphs, register maps, and historical failure signatures for semantic similarity retrieval and institutional memory.

Key Classes & Functions: KnowledgeBase • add_documents() • query_similar() • get_or_create_collection()

Interactions & Dataflow: Provides long-term retrieval-augmented memory to Agent 1 and Agent 2.

■ File Path: verisight/rag/retriever.py

System Role: Semantic Vector Search & Context Ranker

Description & Purpose: Performs cosine similarity search against ChromaDB collections, retrieving only the most relevant specification and historical defect chunks for LLM context.

Key Classes & Functions: RAGRetriever • retrieve_relevant_chunks() • rank_by_similarity()

Interactions & Dataflow: Retrieval helper for agents.

■ File Path: verisight/rag/embeddings.py

System Role: Dense Embedding Model Gateway

Description & Purpose: Generates dense vector embeddings for specifications, register tables, and code snippets.

Key Classes & Functions: EmbeddingEngine • get_embeddings()

Interactions & Dataflow: Embedding generator for ChromaDB.

■ File Path: verisight/rag/train_knowledge_base.py

System Role: Knowledge Base Pre-Indexing Utility

Description & Purpose: Pre-indexes standard protocol specifications (AMBA, I2C, SPI) into ChromaDB for instant zero-shot retrieval during live verification sessions.

Key Classes & Functions: train_knowledge_base() • index_specs()

Interactions & Dataflow: Offline setup utility.

■ File Path: verisight/benchmarks/root_cause_bench.py

System Role: Rigorous Industrial Benchmark Suite Harness

Description & Purpose: Automated benchmarking harness executing 200+ injected failure cases across Arithmetic Logic Units, circular FIFOs, AMBA AXI-Stream, and RISC-V RV32I pipelines. Measured the 14.8s average triage time and 95.4% accuracy.

Key Classes & Functions: RootCauseBench • run_benchmarks() • evaluate_alu() • evaluate_fifo() • evaluate_axi() • evaluate_riscv() • print_scorecard()

Interactions & Dataflow: Benchmark validation infrastructure.